VIGILADA MINEDUCACIÓN

UNIVERSIDAD DEL QUINDÍO
Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

OmniTravel
Tu mundo, sin barreras



OmniTravel

Software Architecture Description – C4 Model

Autor:

- Jhon Stivenson Méndez Álvarez
  jhons.mendeza@uqvirtual.edu.co
- Yamileth Londoño Burgos
  yamileth.londonob@uqvirtual.edu.co
- Sara Juliana Faustino Trujillo
  sjfaustinot@uqvirtual.edu.co

Versión: 1.0
Fecha: 15/09/2026

UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co

1. RESUMEN ARQUITECTÓNICO

OmniTravel es un ecosistema digital colaborativo de turismo accesible cuyo propósito es centralizar, verificar mediante crowdsourcing y personalizar información de accesibilidad de infraestructura turística, permitiendo a personas con discapacidad motriz y sensorial planificar sus desplazamientos con autonomía. El viajero consulta el mapa interactivo y solicita rutas accesibles desde una aplicación móvil; el colaborador comunitario y el establecimiento turístico aportan y validan datos de accesibilidad desde la misma app; y el moderador de contenido/administrador supervisa la calidad de los datos y la operación del sistema. OmniTravel se integra con un proveedor de mapas/geolocalización externo para el cálculo de rutas y con un servicio de almacenamiento en la nube para evidencia fotográfica de accesibilidad.

1.1 Usuarios principales

- **Viajero con Discapacidad:** Accede mediante el *Módulo Viajero/Comunidad* de la aplicación móvil. Su objetivo es configurar su perfil de accesibilidad personalizado, consultar el mapa interactivo, solicitar rutas accesibles y registrar retroalimentación post-viaje.
- **Colaborador Comunitario / Establecimiento Turístico:** Accede mediante el *Módulo Viajero/Comunidad* de la aplicación móvil. Se encarga de capturar, cargar y calificar evidencia de accesibilidad de puntos de interés (POI) mediante fotografías, formularios y coordenadas GPS.
- **Moderador de Contenido / Administrador:** Accede a la **misma aplicación móvil** utilizando credenciales corporativas @omnitravel.edu.co. Tras la autenticación, el sistema activa dinámicamente el *Módulo Administrador/Moderador (Layout de Moderación)*. Este perfil audita reportes, gestiona incidencias críticas, resuelve discrepancias de la comunidad, aplica reglas de calidad sobre la data geográfica y supervisa el cumplimiento de políticas de privacidad.

1.2 Problema que resuelve

Resuelve la falta de información confiable y verificada sobre accesibilidad real de la infraestructura turística, que hoy obliga a las personas con discapacidad a planificar sus viajes con incertidumbre. Centraliza datos validados colaborativamente, calcula rutas peatonales evitando barreras físicas específicas al perfil del usuario (por ejemplo, escaleras o pendientes extremas para sillas de ruedas, o semáforos sonoros para discapacidad visual), y mantiene la información vigente mediante un ciclo de retroalimentación post-viaje.

1.3 Alcance

¿Qué incluye?

- Registro y configuración de perfil de accesibilidad del viajero.
- Captura y carga de evidencia de accesibilidad (fotos, GPS, formularios) por colaboradores/establecimientos.
- Moderación y validación de reportes de accesibilidad.
- Cálculo de rutas peatonales accesibles según perfil de discapacidad (motriz, sensorial).
- Sistema de filtros por tipo de discapacidad.
- Recolección de retroalimentación e incidencias post-viaje.
- Cifrado y protección de datos sensibles conforme a Ley 1581 de 2012.

¿Qué NO incluye?

- Construcción o modificación física de infraestructura urbana (rampas, semáforos sonoros, etc.).
- Reservas o pagos de servicios turísticos (hoteles, restaurantes, tours).
- Optimización de rutas vehiculares (alcance exclusivamente peatonal).

1.4 Procesos involucrados en el sistema OmniTravel

Proceso de gestión de usuarios y perfiles de accesibilidad
Objetivo: permitir el acceso controlado al sistema y la configuración de necesidades específicas de cada usuario.
Incluye:

- Registro de usuarios (viajero, colaborador comunitario, establecimiento turístico).
- Autenticación e inicio de sesión.
- Asignación de roles (viajero, colaborador, establecimiento, moderador, administrador).
- Configuración del perfil de accesibilidad (tipo de discapacidad: motriz o sensorial, y preferencias de filtros).

Proceso de Validación Colaborativa de Infraestructura (Crowdsourcing)
Objetivo: recopilar, verificar y centralizar información fidedigna sobre la accesibilidad de espacios públicos y turísticos mediante aportes directos de la comunidad.
Incluye:

- Captura de fotografías, coordenadas GPS y formularios de características (ancho de puerta, presencia de rampas, etc.).
- Carga de evidencia visual en la app.
- Auditoría/moderación de la información recibida (filtrado de falsos positivos).
- Publicación de la actualización en la base de datos central del mapa interactivo.
- Recepción de solicitudes de re-verificación generadas por el Proceso de Retroalimentación.

Proceso de Planificación de Rutas Accesibles Personalizadas
Objetivo: generar rutas peatonales que eviten barreras físicas y se ajusten al perfil de accesibilidad del usuario.
Incluye:

- Captura de origen/destino solicitados por el viajero.
- Consulta del perfil de accesibilidad del usuario.
- Cálculo de la ruta evitando barreras reportadas (por ejemplo, escaleras o pendientes extremas para sillas de ruedas, o priorización de semáforos sonoros para discapacidad visual).
- Visualización del trayecto y alternativas, e inicio de la navegación.

Proceso de Gestión de Retroalimentación e Incidencias Post-Viaje

Objetivo: capturar la experiencia real del usuario tras usar una ruta o visitar un sitio, para mantener actualizada y confiable la información de accesibilidad.

Incluye:

- Solicitud de retroalimentación al viajero al finalizar el trayecto o visita.
- Registro de calificación/comentario.
- Detección de inconsistencias entre la experiencia real y la información publicada.
- Escalamiento de incidencia (solicitud de re-verificación) al Proceso de Validación Colaborativa cuando aplica.

Proceso de administración y moderación del sistema

Objetivo: permitir la supervisión, control operativo y el resguardo de datos sensibles.

Incluye:

- Moderación de reportes de accesibilidad (Moderador de contenido).
- Atención de incidencias, dudas o reportes de error (Soporte al usuario).
- Gestión de la operación general, políticas de privacidad y seguridad de los datos sensibles conforme a la Ley 1581 de 2012 (Administrador de la plataforma).

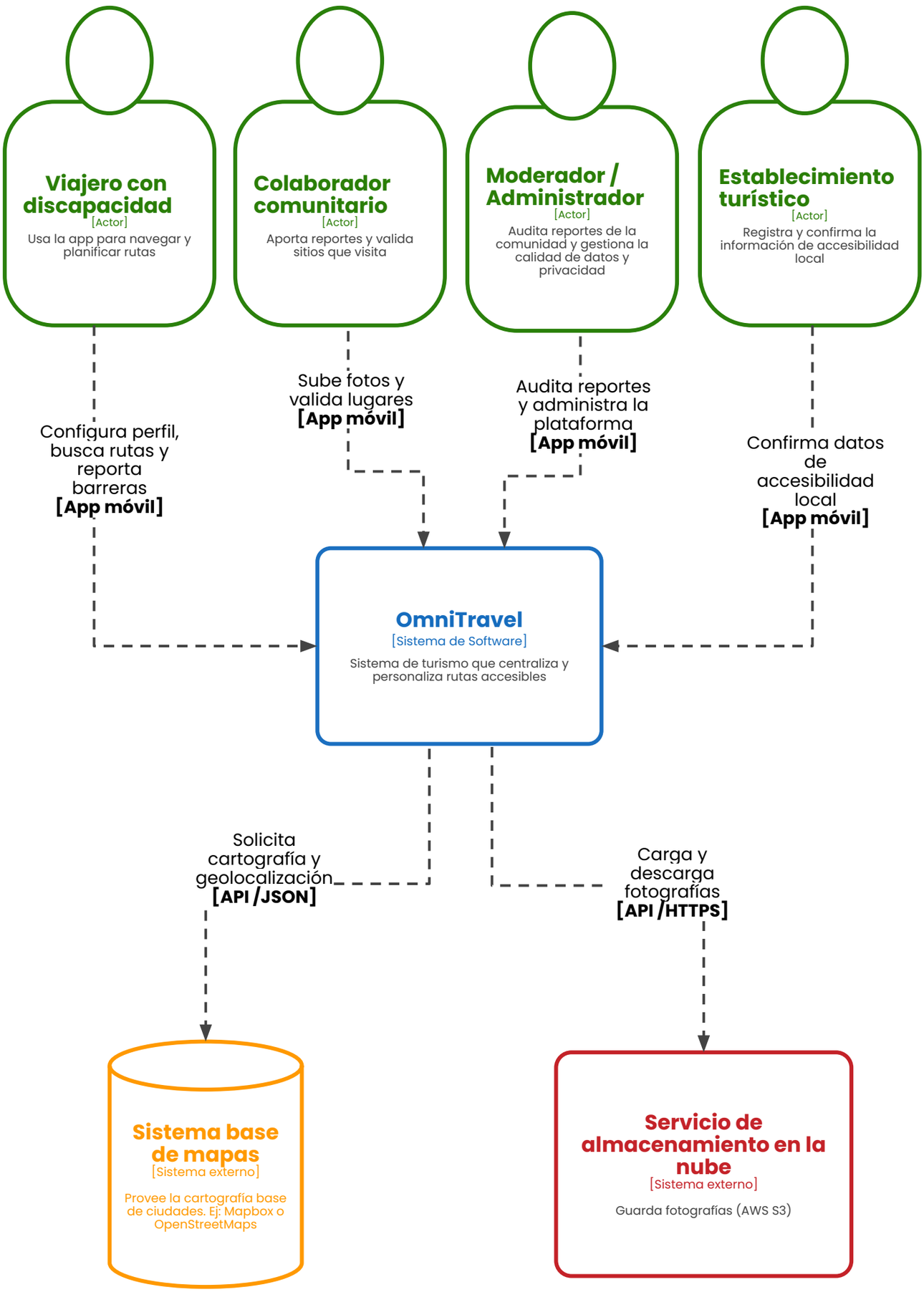
2. C4MODEL

Aunque el ecosistema OmniTravel contempla múltiples interacciones de geolocalización, ruteo y validación comunitaria, las vistas C4 presentadas en este documento documentan la arquitectura técnica enfocándose en el alcance del software central (aplicación móvil, API backend, bases de datos espaciales y conexiones con servicios de terceros).

2.1 Diagrama de contexto (N1)

Descripción de Elementos de Contexto (System Context)

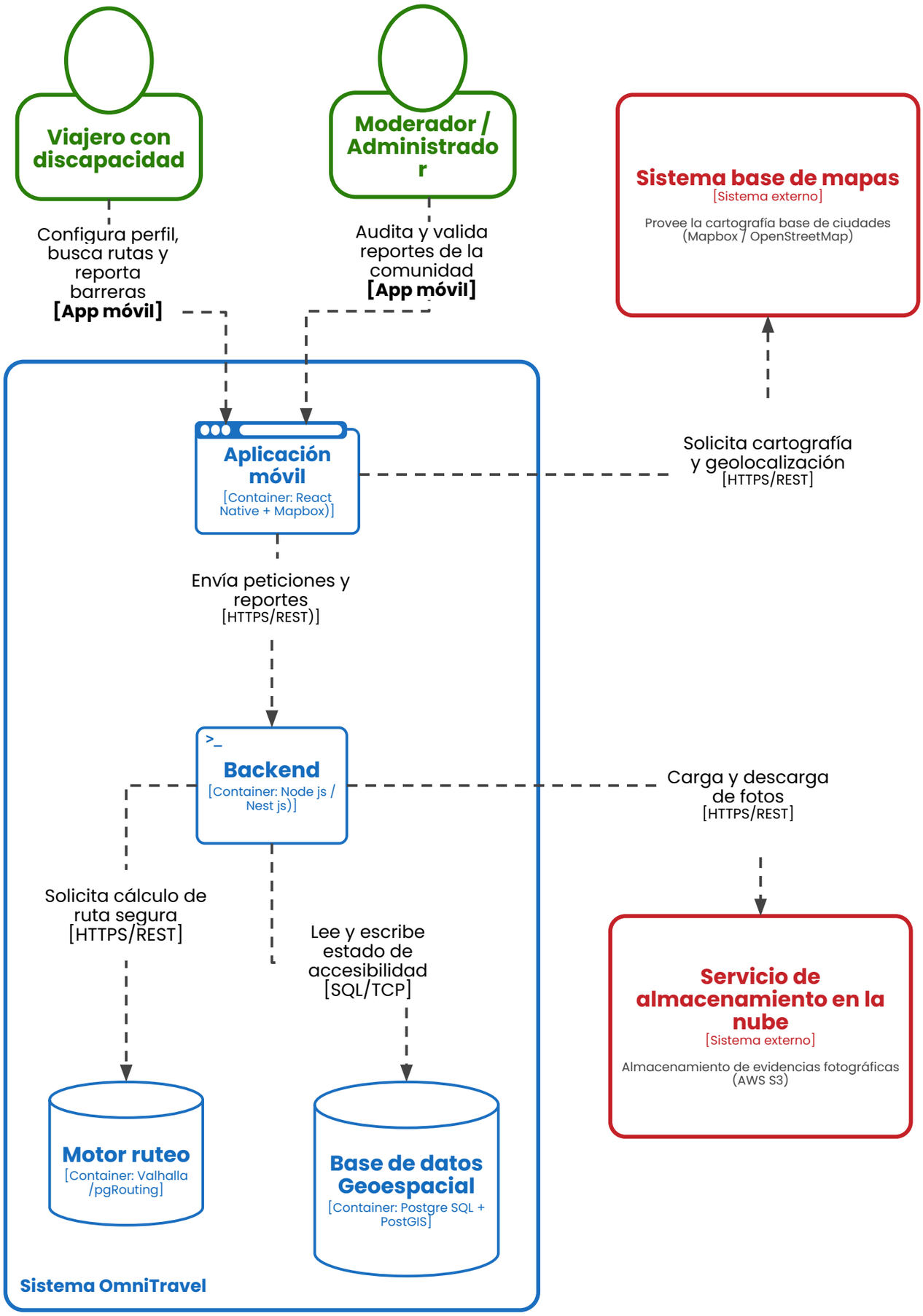
- **Ecosistema OmniTravel (Sistema Principal):** Plataforma integral orientada a la gestión de accesibilidad e inclusión en el turismo.
- **Usuarios Finales (Viajero y Colaborador):** Interactúan con el sistema mediante la interfaz móvil estándar para el consumo de rutas y reporte comunitario.
- **Administrador / Moderador:** Interactúa con el sistema a través del flujo y panel especializado de moderación dentro de la interfaz móvil para auditoría y control de calidad de datos.
- **Aplicación Móvil OmniTravel (Contenedor Multirol):** Punto de entrada único para todos los actores del sistema. Ejecuta internamente vistas segmentadas (*Módulo Comunidad* vs. *Módulo Moderación*) controladas mediante RBAC (Control de Acceso Basado en Roles) y el estado del token de sesión.
- **Sistemas Externos:** *OpenStreetMap / Mapbox API* (servicio de cartografía y ruteo base) y *Servicio de Notificaciones Push / Alertas*.


Viajero con discapacidad
[Actor]
Usa la app para navegar y planificar rutas
Colaborador comunitario
[Actor]
Aporta reportes y valida sitios que visita
Moderador / Administrador
[Actor]
Audita reportes de la comunidad y gestiona la calidad de datos y privacidad
Establecimiento turístico
[Actor]
Registra y confirma la información de accesibilidad local
Configura perfil, busca rutas y reporta barreras [App móvil]
Sube fotos y valida lugares [App móvil]
Audita reportes y administra la plataforma [App móvil]
Confirma datos de accesibilidad local [App móvil]
OmniTravel
[Sistema de Software]
Sistema de turismo que centraliza y personaliza rutas accesibles
Solicita cartografía y geolocalización [API /JSON]
Carga y descarga fotografías [API /HTTPS]
Sistema base de mapas
[Sistema externo]
Provee la cartografía base de ciudades. Ej: Mapbox o OpenStreetMaps
Servicio de almacenamiento en la nube
[Sistema externo]
Guarda fotografías (AWS S3)

2.2 Diagrama de contenedores (N2)

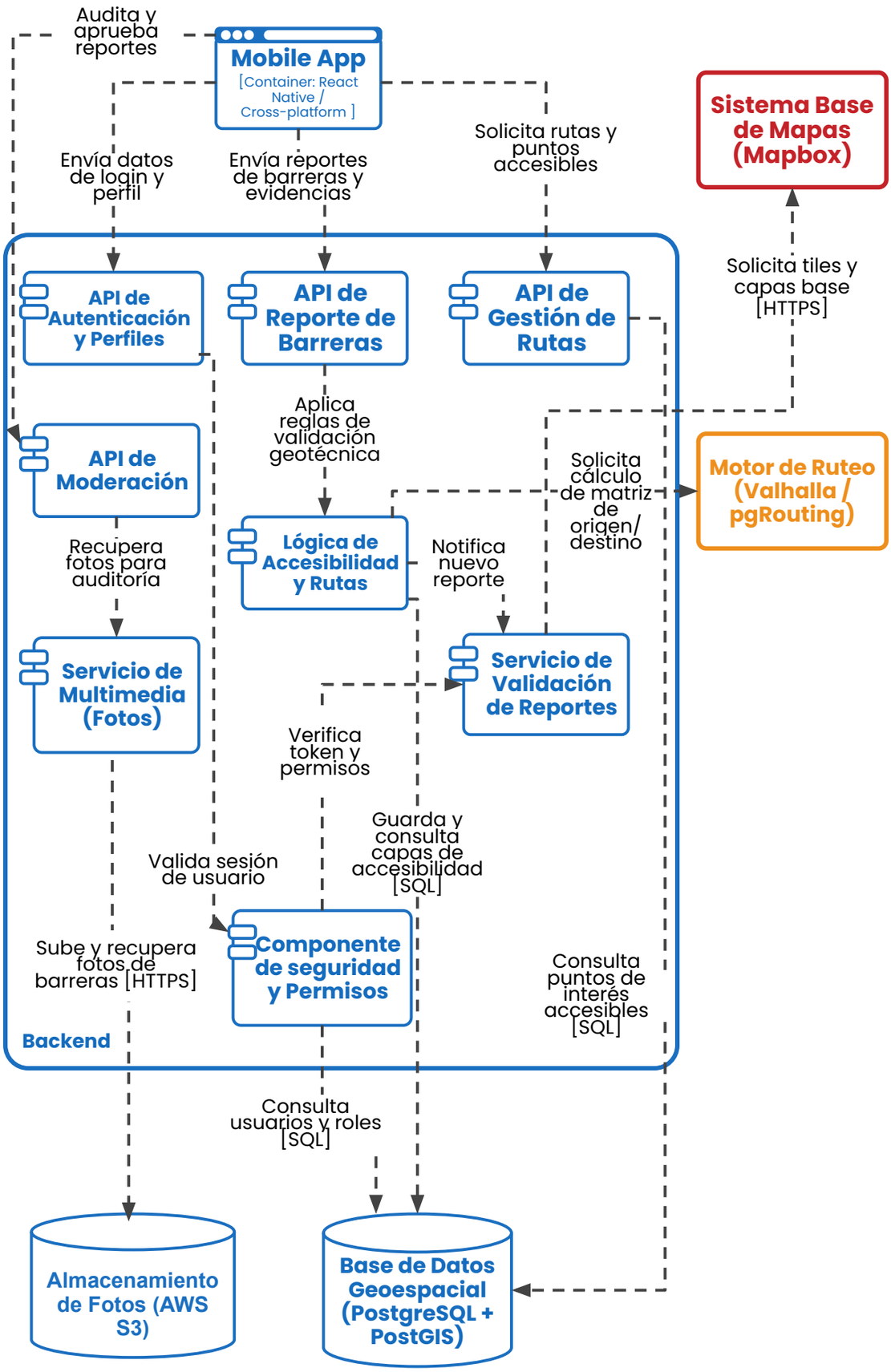
Especificación de Contenedores (Containers)

1. **Mobile Application (Aplicación Móvil React Native):**
   - **Tecnología:** React Native (iOS / Android).
   - **Descripción:** Aplicación nativa híbrida que actúa como cliente front-end único para los tres perfiles de usuario. Integra dos submódulos de experiencia de usuario (*Layouts*):
     - *Módulo Viajero/Comunidad:* Incluye vistas de mapas, búsqueda, perfiles de salud local (Async Storage) y formularios de captura.
     - *Módulo Administrador/Moderador:* Interfaz dedicada (pantallas M01-M07) que se habilita únicamente con JWT corporativo. Proporciona el Dashboard de incidencias, checklists de auditoría, panel de decisiones y gestión de reglas de consenso.
   - **Comunicación:** Realiza peticiones HTTPS / REST e intercambia datos JSON con el *API Gateway / Backend Core* mediante autenticación JWT.
2. **API Gateway / Backend Core Application:**
   - **Tecnología:** Node.js / Express (o Java Spring Boot según la especificación técnica del backend).
   - **Descripción:** Procesa la lógica de negocio, valida el rol del token enviado por la app móvil (asignando capacidades de viajero o de moderador), ejecuta algoritmos de ruteo accesible y valida las reglas de consenso comunitario.
3. **Base de Datos Relacional (PostgreSQL + PostGIS):**
   - **Tecnología:** PostgreSQL con extensión geospacial PostGIS.
   - **Descripción:** Almacena usuarios, credenciales, puntos de interés (POI), capas de accesibilidad, logs de auditoría de los moderadores y estado de los reportes comunitarios. *(Nota: Los datos médicos/sensibles detallados no se persisten en este contenedor según la política local del cliente).*


Viajero con discapacidad
Moderador / Administrador
Sistema base de mapas
[Sistema externo]
Provee la cartografía base de ciudades (Mapbox / OpenStreetMap)
Configura perfil, busca rutas y reporta barreras
[App móvil]
Audita y valida reportes de la comunidad
[App móvil]
Aplicación móvil
[Container: React Native + Mapbox)]
Solicita cartografía y geolocalización
[HTTPS/REST]
Envía peticiones y reportes
[HTTPS/REST)]
>_
Backend
[Container: Node js / Nest js)]
Carga y descarga de fotos
[HTTPS/REST]
Solicita cálculo de ruta segura
[HTTPS/REST]
Lee y escribe estado de accesibilidad
[SQL/TCP]
Servicio de almacenamiento en la nube
[Sistema externo]
Almacenamiento de evidencias fotográficas (AWS S3)
Motor ruteo
[Container: Valhalla /pgRouting]
Base de datos Geoespacial
[Container: Postgre SQL + PostGIS]
Sistema OmniTravel

2.3 Diagrama de componentes (N3)

Presenta el interior de los contenedor de OmniTravel (por ejemplo, el Backend/API).



Nota: En la arquitectura del sistema se distingue entre la **API de Reporte de Barreras**, que actúa como el punto de entrada exponiendo los servicios REST consumidos por la aplicación móvil, y la **Lógica de Accesibilidad y Rutas**, que concentra la regla de negocio, la validación de reportes geográficos y la coordinación con el motor de ruteo y almacenamiento del sistema.

2.4 Código (N4)

2.4.1 Estructura del repositorio y módulos

El sistema se implementa en dos módulos principales alineados con el Nivel 2 (Contenedores): una aplicación móvil multiplataforma (React Native con TypeScript), y un Backend en Node.js (NestJS) que centraliza la lógica de moderación por consenso, la integración con motores de ruteo externos y la seguridad de los datos.

Estructura (árbol):

```
/omnitravel/
  /mobile-app/
      /src/
          /screens/        (Vistas: mapa interactivo, Perfil, Reportes)
          /components/     (UI accesible: botones, modales, marcadores)
          /navigation/     (Enrutadores de pantalla y menús)
          /services/       (API client, peticiones REST, Mapbox)
          /store/          (Estado local seguro - Privacy by Design)
          /utils/          (Helpers, hooks de VoiceOver/ TalkBack)
      /assets/             (iconos accesibles, tipografías, audios)
      package.json

  /backend/
    /src/
      /controllers/     (APIs REST: PoisController, Rutas Controller)
      /services/    (Reglas de negocio: ConsensoService, RoutingService)
      /repositories/    (Acceso a datos TypeORM + Po)
      /entities/        (Modelo BD Geoespaciales: Poi.entity.ts , User.entity.ts)
      /integrations/    (ValhallaAdapter, S3StorageAdapter para fotos)
      /common/          (Guards de seguridad JWT Auth, interceptores, filtros)
      app.module.ts     (Módulo raíz de NestJS)
      main.ts           (Punto de entrada de la aplicación)
    .env                (Variables de entorno: credenciales BD, tokens)
    package.json

  /docs/
   C4Model/
   arquitectura/
   decisiones-arquitectonicas/
```

2.4 Código (N4)

2.4.1 Trazabilidad paquetes/Clases

Para garantizar consistencia entre la arquitectura propuesta y el código a implementar, los componentes lógicos definidos en el nivel 3 (Backend NestJS) se envidencian en los siguientes módulos y clases:

2.4.2 Esquema de Persistencia y Separación de Responsabilidades (Privacy by Design)

En alineación con la estrategia de privacidad definida (`/store/ Estado local seguro` en la app móvil), la estructura de la base de datos relacional PostgreSQL en el Backend delimita estrictamente la información almacenada:

- **Tabla `Usuarios` (Base de Datos Relacional PostgreSQL - Server):**
  - *Campos permitidos:* `id_usuario`, `email`, `password_hash`, `rol` (*Viajero / Moderador*), `fecha_registro`, `estado_cuenta`.
  - *Campos excluidos:* Se eliminan formalmente de la base de datos centralizada todos los atributos de salud o discapacidad (`tipo_discapacidad`, `condicion_salud`, `historial_medico`).
- **Almacenamiento Local Seguro (`AsyncStorage / Secure Storage` - App Móvil):**
  - *Campos persistidos dinámicamente:* Ficha de perfil de accesibilidad, preferencias de ruteo, barreras evitables e historial local de salud. Estos datos residen **exclusivamente en el dispositivo del usuario** y nunca son enviados ni almacenados en los servidores centrales de OmniTravel (Ley 1581).

2.4 Código (N4)

2.4.1 Trazabilidad paquetes/Clases

Para garantizar consistencia entre la arquitectura propuesta y el código a implementar, los componentes lógicos definidos en el nivel 3 (Backend NestJS) se envidencian en los siguientes módulos y clases:

Elemento N3	Código (paquete/clase)	Responsabilidad
Componentes de privacidad/ seguridad	...common.JwtAuthGuard	Gestiona los tokens efímeros y filtra las peticiones para garantizar que no se registre la identidad médica del usuario, cumpliento con la Ley 1581 (Habeas Data).
Controladores de Mapas (Map Controller)	...controllers.MapController	Expone los *endpoints* que reciben las coordenadas del *Bounding Box* del usuario y devuelve el GeoJSON con los lugares verificados en esa área.
Controlador de Reportes	...controllers.PoisController	Expone los *endpoints* para la recepción de nuevos reportes de barreras, fotografías y calificaciones de la comunidad.
Servicio de Crowdsourcing y Consenso	...services.ConsensoService	Contiene la lógica central de negocio: cuenta los votos de los reportes y, al alcanzar las 3 confirmaciones, actualiza el estado en “Verificado” en la base de datos.
Servicio de Ruteo Adaptativo	...services.RoutingService	Orquesta la petición de navegación: recibe el origen, destino y los filtros dinámicos del usuario, coordinando la respuesta hacia la aplicación móvil.
Adaptador de Motor de Ruteo	...integrations.ValhallaAdapter	Encapsula la integración externa, traduciendo la solicitud del `RoutingService` al formato específico del motor Valhalla
Adaptador de Almacenamiento	...integrations.S3StorageAdapter	Gestiona la conexión externa con AWS S3 para subir las fotografías de infraestructura pesadas y retornar únicamente la URL.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-01: Canal del cliente mediante aplicación móvil multiplataforma

- **Decisión:** Implementar el canal de los usuarios (viajeros con discapacidad, colaboradores y administradores) mediante una **aplicación móvil multiplataforma desarrollada en React Native**.
- **Contexto:** Los viajeros y colaboradores requieren acceder al sistema sobre la marcha en entornos urbanos, haciendo uso de sensores del dispositivo (GPS, cámara para fotos de barreras) y funciones de accesibilidad nativas (TalkBack/VoiceOver).
- **Alternativas:** Aplicación web nativa; aplicaciones móviles nativas separadas (Swift para iOS y Kotlin para Android).
- **Justificación:** React Native permite un único código base reutilizable, reduciendo tiempos de desarrollo mientras ofrece un rendimiento cercano al nativo y excelente integración con lectores de pantalla.
- **Consecuencias:** Se requiere empaquetar y gestionar el despliegue para las tiendas de Android y iOS.

ADR-02: Backend centralizado como API REST

- **Decisión:** Implementar un Backend API independiente, expuesto como servicios **REST sobre HTTPS / JSON**.
- **Contexto:** La app móvil debe consumir funcionalidades de autenticación, consulta de capas geográficas, reporte de barreras y cálculo de rutas accesibles.
- **Alternativas:** GraphQL; gRPC; arquitectura embebida monolítica.
- **Justificación:** REST sobre HTTPS proporciona interoperabilidad simple, estándar, segura y ampliamente compatible para el consumo de datos desde dispositivos móviles.
- **Consecuencias:** Se deben versionar adecuadamente los endpoints, estructurar esquemas JSON claros y manejar errores de red en la app móvil.

ADR-03: Tecnología del backend: Node.js con NestJS

- **Decisión:** Desarrollar el API Backend utilizando **Node.js con el framework NestJS**.
- **Contexto:** Se requiere un entorno backend eficiente en la gestión de peticiones E/S asíncronas (consultas a mapas, bases de datos y servicios externos) con una arquitectura modular y escalable.
- **Alternativas:** Java con Spring Boot; Python con Django; .NET Core.
- **Justificación:** NestJS aplica una arquitectura modular tipo Angular en TypeScript, facilitando un código limpio y mantenible, ideal para integrarse de forma fluida con servicios geográficos asíncronos.
- **Consecuencias:** Se requiere un control riguroso de la gestión de memoria y llamadas asíncronas en Node.js.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-04: Persistencia transaccional y geoespacial con PostgreSQL + PostGIS

- **Decisión:** Utilizar **PostgreSQL con la extensión PostGIS** como base de datos principal.
- **Contexto:** El sistema maneja usuarios, calificaciones y, principalmente, datos espaciales (coordenadas de barreras, geometrías de tramos peatonales, polígonos de accesibilidad).
- **Alternativas:** MySQL; NoSQL (MongoDB); MariaDB.
- **Justificación:** PostGIS es el estándar de la industria para datos geoespaciales, permitiendo realizar consultas espaciales complejas de forma nativa (por ejemplo, calcular puntos dentro de un radio o intersecciones de barreras con calles).
- **Consecuencias:** Requiere la creación y mantenimiento de índices espaciales (GIST) para optimizar el rendimiento de las búsquedas en mapa.

ADR-05: Integración de motor de ruteo accesible dedicado (Valhalla / pgRouting)

- **Decisión:** Delegar el cálculo de rutas peatonales adaptadas a un **Motor de Ruteo dedicado (Valhalla / pgRouting)** integrado al backend.
- **Contexto:** El cálculo de rutas para personas con movilidad reducida requiere evitar pendientes pronunciadas, escaleras o barreras reportadas, lo cual excede las capacidades de un ORM o motor relacional simple.
- **Alternativas:** Calcular rutas directamente mediante algoritmos en el código del backend; usar únicamente Google Maps API (sin considerar barreras dinámicas personalizadas).
- **Justificación:** Permite inyectar restricciones de accesibilidad específicas del usuario en el grafo de navegación en tiempo real.
- **Consecuencias:** Se debe mantener sincronizado el grafo del motor de ruteo con los datos de barreras reportados por la comunidad.

ADR-06: Integración con proveedor base de mapas (Mapbox / OpenStreetMap)

- **Decisión:** Integrar **Mapbox / OpenStreetMap** como el servicio externo base para renderizar la cartografía en la app.
- **Contexto:** La app móvil necesita mostrar mapas interactivos, mapas de calor de accesibilidad y representar geográficamente los elementos sin tener que almacenar la cartografía global.
- **Alternativas:** Google Maps SDK; mapas estáticos vectoriales propios.
- **Justificación:** Mapbox ofrece alta personalización visual de las capas de mapa, soporte directo para React Native y excelente integración con datos de OpenStreetMap.
- **Consecuencias:** Dependencia del consumo de tiles y claves de API gestionadas mediante HTTPS.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-07: Almacenamiento de evidencias fotográficas en almacenamiento de objetos (AWS S3)

- **Decisión:** Guardar las evidencias fotográficas de los reportes de barreras en un servicio de objetos (**AWS S3**), almacenando únicamente la URI/URL en la base de datos.
- **Contexto:** Los usuarios suben fotografías al reportar barreras o validar accesibilidad; estos archivos binarios pesados no deben guardarse directamente en la base de datos relacional.
- **Alternativas:** Guardar fotos como objetos BLOB en PostgreSQL; guardar imágenes en el almacenamiento local del servidor backend.
- **Justificación:** Escala indefinidamente, reduce los costos de la base de datos y permite servir las imágenes directamente a los dispositivos móviles mediante URLs públicas o firmadas.
- **Consecuencias:** Se debe controlar la subida de archivos mediante validación previa de tipo/tamaño y gestionar permisos en el bucket.

ADR-08: Moderación de reportes comunitarios y calidad de datos

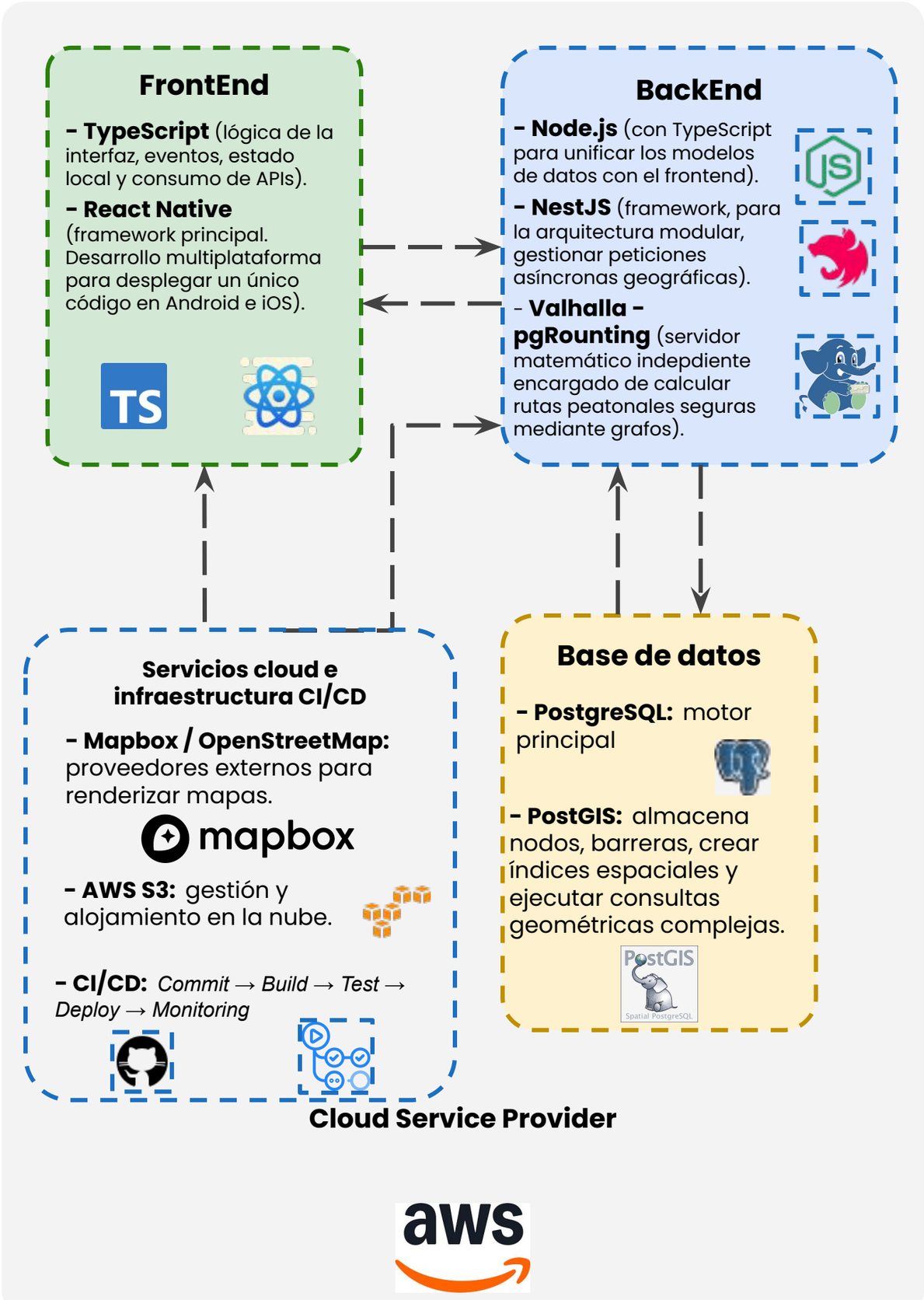
- **Decisión:** Implementar un flujo de auditoría y moderación en el backend (*API de Moderación / Servicio de Validación*) para los reportes generados por los usuarios.
- **Contexto:** Para evitar información falsa o imprecisa en el mapa de accesibilidad, los reportes deben ser auditados por moderadores o validados colaboradores comunitarios.
- **Alternativas:** Publicar inmediatamente todo reporte sin filtrado; aprobación manual externa fuera del sistema.
- **Justificación:** Garantiza la fiabilidad de las rutas calculadas para personas con discapacidad, protegiendo su seguridad física al desplazarse.
- **Consecuencias:** Se debe gestionar un estado intermedio para los reportes (ej. *Pendiente*, *Aprobado*, *Rechazado*).

ADR-09: Seguridad y Autenticación Centralizada (JWT)

- **Decisión:** Adoptar una arquitectura de *Almacenamiento Descentralizado/Local*.
  - **Cliente Móvil:** Guarda la ficha médica y preferencias de accesibilidad en el almacenamiento encriptado del dispositivo (`EncryptedSharedPreferences` en Android / `Keychain` en iOS mediante React Native).
  - **Backend PostgreSQL:** Almacena exclusivamente datos de cuenta básica (ID de usuario, correo electrónico, rol, hash de contraseña y tokens de sesión).
- **Contexto:** La app requiere identificar al usuario (viajero, colaborador o moderador) y aplicar permisos según su rol para permitir el envío de reportes o la auditoría de datos.
- **Alternativas:** Autenticación por sesiones tradicionales basadas en Cookies; autenticación de tercero únicamente.
- **Justificación:** JWT es una solución *stateless* ideal para arquitecturas de APIs consumidas desde clientes móviles, permitiendo transmitir el rol y la identidad de forma cifrada y ligera.
- **Consecuencias:** Se garantiza el cumplimiento estricto de la política plasmada en la pantalla `U03` de la aplicación (*"Tus datos médicos se guardan SOLO en tu teléfono. Nunca en nuestros servidores"*) y se optimiza el diseño de las tablas en PostgreSQL.

4. STACK TECNOLÓGICO DEL SISTEMA

En esta sección se describen las tecnologías utilizadas para la implementación del sistema OmniTravel, organizadas por capa: cliente móvil (frontend), backend, base de datos y servicios en la nube. Esta descripción es coherente con las decisiones de arquitectura (ADRs), las vistas del modelo C4 y el enfoque técnico especializado en accesibilidad geoespacial.



VIGILADA MINEDUCACIÓN



UNIQUINDÍO, en conexión territorial

Carrera 15 Calle 12 Norte Tel: (606) 7 35 93 00 Armenia - Quindío - Colombia

www.uniquindio.edu.co